

Manual imprimible

Hugging Face Spaces con Docker

Gradio y Streamlit

Versión para estudio, práctica y consulta rápida

Este manual explica cómo publicar aplicaciones en Hugging Face Spaces usando Docker, tanto con Gradio como con Streamlit. Incluye estructura de archivos, ejemplos mínimos, plantillas completas, errores comunes y fuentes oficiales.

1. Idea general

Hugging Face Spaces permite publicar aplicaciones web, demos de modelos y proyectos interactivos. Un Space puede usar distintos SDK. Para aplicaciones simples se puede usar Gradio directamente. Para Streamlit, Hugging Face actualmente indica usar Docker. Docker también sirve cuando se necesita más control sobre el entorno.

```
---
title: Mi app Docker
sdk: docker
app_port: 7860
---
```

Con Docker, Hugging Face construye la aplicación a partir de un Dockerfile. Esto permite elegir versión de Python, instalar paquetes del sistema, fijar dependencias y definir el comando de arranque de la app.

2. Cuándo conviene usar Docker

Situación	Recomendación
App simple de Gradio	No necesariamente. Puede alcanzar con sdk: gradio.
App de Streamlit	Sí. Hugging Face indica usar Docker para Streamlit.
Necesitás una versión específica de Python	Sí.
Necesitás instalar paquetes del sistema	Sí.
Usás OpenCV, ffmpeg, Tesseract, MediaPipe u otras dependencias especiales	Sí.
Querés reproducibilidad y control del entorno	Sí.
Estás empezando con una demo muy simple	Podés empezar sin Docker, salvo Streamlit.

3. Estructura general de un Docker Space

```
mi-space/
|
|-- README.md
|-- Dockerfile
|-- requirements.txt
`-- app.py
```

Archivo	Función
README.md	Configura el Space mediante un bloque YAML al inicio.
Dockerfile	Indica cómo construir el contenedor.
requirements.txt	Lista las librerías de Python.

4. Tutorial: Gradio con Docker

4.1. Archivos necesarios

```
mi-space-gradio-docker/  
|  
|-- README.md  
|-- Dockerfile  
|-- requirements.txt  
`-- app.py
```

4.2. README.md

```
---  
title: Gradio con Docker  
emoji: 🐳  
colorFrom: blue  
colorTo: gray  
sdk: docker  
app_port: 7860  
---
```

Gradio con Docker en Hugging Face Spaces

Aplicación básica de Gradio ejecutada en un Docker Space.

Lo importante es que el SDK sea docker y que el puerto declarado coincida con el puerto usado por la aplicación.

4.3. requirements.txt

```
gradio
```

Para una app con imágenes se pueden agregar otras dependencias:

```
gradio  
numpy  
pillow  
opencv-python-headless
```

4.4. app.py básico con Gradio

```
import gradio as gr  
  
def saludar(nombre):  
    return f"Hola, {nombre}. Esta app corre con Gradio + Docker en Hugging Face Spaces."  
  
demo = gr.Interface(  
    fn=saludar,  
    inputs=gr.Textbox(label="Nombre"),  
    outputs=gr.Textbox(label="Respuesta"),  
    title="Demo Gradio con Docker"  
)  
  
demo.launch()
```

```

server_name="0.0.0.0",
server_port=7860
)

```

El valor `server_name="0.0.0.0"` permite exponer la aplicación desde dentro del contenedor. No conviene usar solamente `localhost` o `127.0.0.1`.

4.5. Dockerfile para Gradio

```

FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 7860

CMD ["python", "app.py"]

```

4.6. Checklist Gradio

Lugar	Debe tener
README.md	sdk: docker
README.md	app_port: 7860
Dockerfile	EXPOSE 7860
app.py	server_name="0.0.0.0"
app.py	server_port=7860

5. Tutorial: Streamlit con Docker

Streamlit suele correr en el puerto 8501. En Hugging Face Spaces, para Streamlit conviene usar Docker y declarar ese puerto en el README.md.

5.1. Archivos necesarios

```

mi-space-streamlit-docker/
|
|-- README.md
|-- Dockerfile
|-- requirements.txt
`-- app.py

```

5.2. README.md

```

---
title: Streamlit con Docker
emoji: 🐳
colorFrom: green
colorTo: gray
sdk: docker
app_port: 8501
---

```

Streamlit con Docker en Hugging Face Spaces

Aplicación básica de Streamlit ejecutada en un Docker Space.

5.3. requirements.txt

```
streamlit
```

Para una app con imágenes:

```
streamlit
numpy
pillow
opencv-python-headless
```

5.4. app.py básico con Streamlit

```
import streamlit as st

st.title("Demo Streamlit con Docker")

nombre = st.text_input("Escribí tu nombre")

if nombre:
    st.write(f"Hola, {nombre}. Esta app corre con Streamlit + Docker en Hugging Face Spaces.")
else:
    st.write("Ingresá un nombre para probar la app.")
```

5.5. Dockerfile para Streamlit

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8501

CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

5.6. Checklist Streamlit

Lugar	Debe tener
README.md	sdk: docker
README.md	app_port: 8501
Dockerfile	EXPOSE 8501
Dockerfile / CMD	--server.port=8501
Dockerfile / CMD	--server.address=0.0.0.0

6. Cómo subir el proyecto a Hugging Face Spaces

6.1. Desde la web

1. Crear un nuevo Space en Hugging Face.
2. Elegir Docker como SDK.
3. Subir README.md, Dockerfile, requirements.txt y app.py.
4. Esperar la construcción del Space.
5. Revisar los logs si aparece un error.

6.2. Desde la computadora local con Git

```
git clone https://huggingface.co/spaces/USUARIO/NOMBRE_DEL_SPACE
cd NOMBRE_DEL_SPACE
```

Copiar dentro de esa carpeta los archivos del proyecto. Luego ejecutar:

```
git add .
git commit -m "Agregar app con Docker"
git push
```

Hugging Face detecta el cambio, reconstruye el contenedor y ejecuta la aplicación.

7. Ejemplo con dependencias del sistema

Si la app necesita paquetes de Linux, se instalan en el Dockerfile con apt-get. Esto puede servir para OpenCV, ffmpeg, procesamiento de imágenes, video u OCR.

```
FROM python:3.10-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    ffmpeg \
    libgl1 \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 7860

CMD ["python", "app.py"]
```

8. Errores comunes

Problema	Solución
El Space queda en Building o Runtime error	Revisar logs, requirements.txt, Dockerfile y errores de Python.
La app no abre	Verificar que el puerto coincida en README.md, Dockerfile y app.py/CMD.
Usar localhost	Cambiar por 0.0.0.0.
Dockerfile mal nombrado	Debe llamarse exactamente Dockerfile.
Usar cv2.imshow()	En Spaces se muestra la imagen con Gradio, Streamlit, PIL o Matplotlib.
Usar opencv-python en servidor	Puede convenir opencv-python-headless si no se abren ventanas.

9. Comparación rápida

Tema	Gradio con Docker	Streamlit con Docker
Mejor para	Demos de modelos de IA	Dashboards y apps interactivas
Puerto típico	7860	8501
Archivo principal	app.py	app.py

Comando final	python app.py	streamlit run app.py
Complejidad	Baja	Baja a media

10. Plantilla final mínima: Gradio

README.md
Dockerfile
requirements.txt
app.py

README.md

```
---
title: Gradio Docker App
sdk: docker
app_port: 7860
---

# Gradio Docker App
```

requirements.txt

```
gradio
```

app.py

```
import gradio as gr

def responder(texto):
    return f"Recibí: {texto}"

demo = gr.Interface(
    fn=responder,
    inputs="text",
    outputs="text",
    title="Gradio Docker App"
)

demo.launch(server_name="0.0.0.0", server_port=7860)
```

Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 7860

CMD ["python", "app.py"]
```

11. Plantilla final mínima: Streamlit

README.md
Dockerfile
requirements.txt
app.py

README.md

```
---  
title: Streamlit Docker App  
sdk: docker  
app_port: 8501  
---
```

```
# Streamlit Docker App
```

requirements.txt

```
streamlit
```

app.py

```
import streamlit as st  
  
st.title("Streamlit Docker App")  
  
texto = st.text_input("Escribí algo")  
  
if texto:  
    st.write(f"Recibí: {texto}")
```

Dockerfile

```
FROM python:3.10-slim  
  
WORKDIR /app  
  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
COPY . .  
  
EXPOSE 8501  
  
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

12. Resumen final

Para usar Hugging Face Spaces con Docker, el proyecto necesita README.md, Dockerfile, requirements.txt y app.py. En README.md se declara sdk: docker y app_port. Para Gradio suele usarse el puerto 7860. Para Streamlit suele usarse el puerto 8501. Docker conviene cuando se necesita controlar el entorno, fijar versiones, instalar dependencias del sistema o trabajar con una aplicación más reproducible.

13. Fuentes

- <https://huggingface.co/docs/hub/spaces>

- <https://huggingface.co/docs/hub/spaces-overview>
- <https://huggingface.co/docs/hub/spaces-sdks-docker>
- <https://huggingface.co/docs/hub/spaces-sdks-docker-first-demo>
- <https://huggingface.co/docs/hub/spaces-sdks-gradio>
- <https://huggingface.co/docs/hub/spaces-sdks-streamlit>
- <https://huggingface.co/docs/hub/spaces-changelog>